

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

Reverse Engineering Task

Code of Conduct:

I _____S.Poojitha(192320076)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Poojitha

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

Ans) Legacy E-Learning Management System

1. Main Classes

Class	Attributes	Methods
Student	studentId, name, email	register(), enrollCourse(), submitAssignment()
Course	courseId, title	addContent(), enrollStudent()
Assignment	assignmentId, deadline	submit(), evaluate()
Result	resultId, marks, grade	calculateGrade(), generateResult()
Instructor	instructorId, name	createCourse(), evaluateAssignment()

2. Relationships

- **Student** → **Course**: Student enrolls in courses.
- **Course** → **Content/Assignment**: Course contains content and assignments.
- **Instructor** → **Course**: Instructor manages courses.
- **Assignment** → **Result**: Evaluation produces results.

3. Layered Redesign

Presentation → **Business** → **Domain** → **Data Access** → **Database**

- **Presentation**: User interface for students and instructors.
- **Business**: Handles registration, enrollment, evaluation, and results.
- **Domain**: Contains Student, Course, Assignment, Result, etc.
- **Data Access**: Stores and retrieves data from the database.

4. Benefits

- **Encapsulation**: Protects object data.

- **High Cohesion:** Each class performs a specific responsibility.
- **Low Coupling:** Changes in one layer have less effect on others.
- **Maintainability:** Modules can be modified independently.
- **Scalability:** New features such as online exams and certificates can be added easily.

Conclusion

The redesigned system uses object-oriented principles and layered architecture to replace tightly coupled modules with organized and independent components. This improves cohesion, reduces coupling, and makes the system easier to maintain and scale.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

Ans) Legacy Customer Support System

1. Existing Design Limitations

The system uses many if-else/switch statements to handle email, chatbot, telephone, and social media requests. This causes:

- High coupling between support channels and business logic.
- Difficult maintenance when adding a new channel.
- Repeated and complex conditional code.
- Poor flexibility and violation of the Open/Closed Principle.

2. Refactoring Using Behavioral Pattern

The Strategy Pattern is suitable for this system. Create a common interface:

SupportChannel → **handleRequest()**

Different classes implement it:

- EmailSupport

- ChatbotSupport
- TelephoneSupport
- SocialMediaSupport

The system selects the required support strategy without changing the main support logic.

3. Object-Oriented Structure

CustomerRequest → SupportService → SupportChannel

SupportService uses the selected SupportChannel to process the request. Each channel is responsible only for its own communication method.

4. Benefits

- **Flexibility:** Different support channels can be selected easily.
- **Extensibility:** New channels can be added without modifying existing classes.
- **Loose Coupling:** SupportService does not depend on specific channel implementations.
- **Maintainability:** Each channel has separate and simple logic.
- **Reusability:** Support channel classes can be reused in other applications.

Conclusion

Replacing conditional statements with the Strategy Pattern creates a cleaner object-oriented design. The system achieves low coupling, better maintainability, and easy extension, allowing future channels such as WhatsApp or mobile-app support to be added with minimal changes.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

Ans) Legacy Banking Transaction System

1. Reverse Engineering – Main Components

Domain Classes:

- **Account:** accountId, balance → deposit(), withdraw()

- **Customer:** customerId, name → updateDetails()
- **Transaction:** transactionId, amount, type → validate()
- **Loan:** loanId, amount, interest → calculateInterest()

Service Classes:

- **AccountService:** manages account operations.
- **TransactionService:** validates and processes transfers.
- **LoanService:** handles loan processing.
- **ReportService:** generates transaction reports.

Data Access Components:

- **AccountRepository:** stores/retrieves account objects.
- **TransactionRepository:** manages transaction data.
- **LoanRepository:** manages loan data.

2. Redesigned Architecture

Presentation → Service → Domain → Repository/DAO → Database

- **Presentation Layer:** Handles user requests.
- **Service Layer:** Contains banking business logic.
- **Domain Layer:** Contains Account, Customer, Transaction, and Loan objects.
- **Data Access Layer:** Uses repositories/DAO for database operations.

3. Suitable Design Patterns

- **Repository Pattern:** Separates database operations from business logic.
- **Factory Pattern:** Creates different account or transaction objects.
- **Strategy Pattern:** Supports different loan-interest or transaction-validation policies.

4. Benefits

- **Separation of Concerns:** Each layer has a clear responsibility.
- **Testability:** Business services can be tested without connecting to the database.
- **Low Coupling:** Changes in database or UI have minimal effect on business logic.
- **Maintainability:** Individual classes and services can be modified independently.
- **Scalability:** New banking features can be added without major changes.

Conclusion

The redesigned layered architecture separates domain logic, business services, and database operations. Using Repository, Factory, and Strategy patterns reduces coupling and improves testability, maintainability, flexibility, and scalability of the banking system.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO**,

Repository, or Data Access patterns, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

Ans) Legacy Hotel Reservation System

1. Problems in Existing Architecture

- **High Coupling:** Reservation screens are directly connected to the OODBMS.
- **Code Duplication:** The same queries for rooms, guests, bookings, and payments are repeated.
- **Poor Persistence Management:** Database operations are mixed with reservation logic.
- **Difficult Maintenance:** Any database change requires modifications in many modules.

2. Reverse Engineering – Main Components

Domain Classes:

- **Guest:** guestId, name, contact
- **Room:** roomId, type, status
- **Booking:** bookingId, checkIn, checkOut
- **Payment:** paymentId, amount, status

Data Access Components:

- GuestDAO
- RoomDAO
- BookingDAO
- PaymentDAO

These components handle all OODBMS operations for their respective objects.

3. Redesigned Architecture

Presentation → Service → Domain → DAO/Repository → OODBMS

- **Presentation:** Handles reservation screens.
- **Service:** Manages booking, cancellation, and payment logic.
- **Domain:** Contains Guest, Room, Booking, and Payment objects.
- **DAO/Repository:** Performs object storage and retrieval from the OODBMS.

4. Benefits

- **Modularity:** Database logic is separated from business logic.

- **Reusability:** DAO methods can be reused by multiple services.
- **Low Coupling:** UI and business classes do not directly depend on the OODBMS.
- **Maintainability:** Database changes are mainly handled within the data access layer.
- **Efficient OODBMS Integration:** Persistent hotel objects can be stored and retrieved through a consistent interface.

Conclusion

Using DAO and Repository patterns removes duplicated database operations and separates persistence from business logic. The redesigned architecture provides better modularity, reusability, maintainability, and cleaner integration with the OODBMS.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

Ans) Legacy Transportation Management System

1. Reverse Engineering – Common Classes

Common Abstraction:

- **Transport:** vehicleId, capacity, fare → book(), calculateFare()

Transport Types:

- **Bus**
- **Train**
- **Taxi**
- **RentalVehicle**

Each type inherits from the Transport class and provides its own implementation.

2. Object Relationships

Transport

- Bus
- Train
- Taxi
- RentalVehicle

The client application uses the common Transport interface instead of directly creating individual objects.

3. Factory Method Pattern

A **TransportFactory** is responsible for creating the required transport object:

Client → **TransportFactory** → **Bus/Train/Taxi/RentalVehicle**

Instead of using repeated new Bus(), new Train(), etc., the application requests the required object from the factory.

4. Benefits

- **Extensibility:** New services such as metro or electric vehicles can be added easily.
- **Code Reuse:** Common transport operations are defined once in the base class.
- **Low Coupling:** Client code does not depend on specific transport classes.
- **Maintainability:** Object creation logic is centralized.
- **Scalability:** New transportation types can be introduced without modifying many existing modules.

Conclusion

The Factory Method Pattern removes duplicated object-creation logic and provides a common abstraction for transportation services. This results in a flexible, reusable, maintainable, and easily extensible transportation management system.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design

patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

Ans) Legacy Healthcare Monitoring System

1. SOLID Violations

- **SRP:** One class performs patient management, sensor processing, alerts, storage, and reporting. It has too many responsibilities.
- **OCP:** Adding a new sensor or alert type requires modifying existing classes.
- **DIP:** Business classes directly depend on concrete sensors and storage implementations.

2. Reverse Engineering – Main Classes

Patient: patientId, name → updateDetails()

Sensor: sensorId, type → readData()

AlertService: generateAlert()

MedicalRecord: recordId, readings → saveRecord()

ReportService: generateReport()

3. Redesigned Structure

Use interfaces to reduce dependencies:

- ISensor → TemperatureSensor, HeartRateSensor
- IStorage → DatabaseStorage, CloudStorage
- IAlert → SMSAlert, EmailAlert

MonitoringService depends on ISensor, IStorage, and IAlert rather than concrete classes.

4. Design Patterns

- **Strategy Pattern:** Supports different sensor-processing or alert algorithms.
- **Factory Pattern:** Creates appropriate sensor objects.
- **Repository Pattern:** Separates medical-record persistence from business logic.

5. Benefits

- **High Cohesion:** Each class performs one clear responsibility.
- **Loose Coupling:** Interfaces separate business logic from devices and storage.
- **Flexibility:** New sensors and storage systems can be added easily.
- **Testability:** Interfaces allow mock sensors and storage during testing.
- **Maintainability:** Changes in one component have minimal effect on others.

Conclusion

Applying SRP, OCP, and DIP with interfaces and suitable design patterns transforms the tightly coupled healthcare system into a modular, flexible, testable, and maintainable application. New monitoring devices and services can be introduced without major changes to existing business logic.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation